Лабораторная работа №5. Линейная регрессия с L1-регуляризацией: SIMD и OpenMP

Поздняков Илья, группа 11-401

1. Стенд и параметры

Intel Xeon Processor @ 2.80 ГГц (виртуальная машина), 2 ядра без SMT, AVX2/AVX-512, L1d 32 КБ и L2 1 МБ на ядро, L3 33 МБ; GCC 13.3.0, основная сборка `gcc -O3 -march=native -fopenmp linreg_simd.c -o linreg_simd -lm`. Данные: $X_{ij} \sim U(-1, 1)$, $y = Xw^* + 0{,}5 +$ шум, у w^* каждый 4-й коэффициент равен 0; ГСЧ с фиксированным seed = 42. Обучение: $w = 0$, $b = 0$, $\eta = 0{,}5$, $\lambda = 10^{-3}$, $K = 100$ итераций, double. Время — только обучение (без генерации, выделения памяти и вывода); прогрев + 5 замеров (для $N = 10^6$ — 3), медиана; серия повторена 3 раза, в таблицах медиана. Корректность: $\Delta w < 10^{-9}$ и $|\Delta F| < 10^{-9}$ относительно serial — выполнено во всех запусках.

2. Результаты

Табл. 1. Корректность ($N = 10^4$, допуск 10^{-9})

Версия	F(w, b)	Δw	OK
Последоват.	0.0107533275	0	да
OpenMP	0.0107533275	6.9e-18	да
SIMD	0.0107533275	2.8e-17	да
OpenMP+SIMD	0.0107533275	5.6e-17	да

Табл. 2. Сравнение ($N = 10^5$, $d = 32$, 2 потока)

Версия	Время, мс	S
Последоват.	361.4	1.00
OpenMP	266.4	1.36
SIMD	288.7	1.25
OpenMP+SIMD	236.8	1.53

Табл. 3. Время (мс) от N (d = 32, 2 потока)

N	serial	OpenMP	SIMD	OMP+SIMD
10^4	24.1	22.1	13.0	16.7
10^5	361.4	266.4	288.7	236.8
10^6	4266	2966	3441	2534

Табл. 4. Время (мс) от d ($N = 10^5$, 2 потока)

d	serial	OpenMP	SIMD	OMP+SIMD	serial/SIMD
8	59.8	34.2	56.8	33.9	1.05
32	361.4	266.4	288.7	236.8	1.25
128	1924	1184	1278	832.7	1.51
256	4391	2429	2793	1632	1.57

Табл. 5. Число потоков ($N = 10^5$, $d = 32$; база — 1 поток той же версии)

p	OpenMP, мс	S	E	OMP+SIMD, мс	S	E
1	400.2	1.00	1.00	307.5	1.00	1.00
2	266.4	1.50	0.75	236.8	1.30	0.65
4	258.8	1.55	0.39	204.7	1.50	0.38
8	229.0	1.75	0.22	186.1	1.65	0.21
16	254.1	1.58	0.10	182.4	1.69	0.11

Табл. 6. Регуляризация (OpenMP+SIMD, $|w_j| < 10^{-6}$)

λ	F(w, b)	Нулевых w	Время, мс
0	0.000417	0 / 32	243.8
10^{-4}	0.001455	8 / 32	229.8
10^{-3}	0.010762	8 / 32	236.8
10^{-2}	0.100636	8 / 32	245.1
10^{-1}	0.718044	18 / 32	237.7

Табл. 7. Флаги компиляции ($N = 10^5$, $d = 32$, 2 потока)

Флаги	Время, мс	S	Примечание
-O0 -fopenmp	1798	1.00	serial, без векторизации
-O3 -fopenmp	534.4	3.36	serial, SSE2 (16 байт)
-O3 -march=native -fopenmp	361.4	4.97	serial, AVX2 (32 байта)
-O3 -march=native -fopenmp + omp simd	288.7	6.23	версия SIMD

Векторизация (-fopt-info-vec, -O3 -march=native). Автоматически векторизованы (32-байтные векторы AVX2, 4 double): цикл градиента g[j] += e·x[j], деление g[j] /= N, мягкое отсечение S(z) (через маски) и скалярное произведение x·w в serial — но только как «упорядоченная» редукция: умножения векторные, сложения идут по одному, т. к. без -ffast-math порядок сложения менять нельзя. Не векторизованы внешние циклы по объектам («loop nest containing two or more consecutive inner loops»), сложение буферов потоков (шаг stride между потоками, «control flow in loop») и цикл расчёта потерь. С -O3 без -march используются лишь 16-байтные SSE2-векторы, и S(z) не векторизуется. omp simd reduction(+:pred) разрешает переставлять сложения: x·w считается в 4 независимых частичных суммах, что и даёт выигрыш версии SIMD.

3. Выводы

1. Самой быстрой почти везде оказалась OpenMP+SIMD: при $N = 10^5$ S = 1,53, при $N = 10^6$ S = 1,68 относительно serial. Исключение — малая задача $N = 10^4$: там быстрее всех чистый SIMD (13 мс против 24 мс у serial), а OpenMP почти не ускоряет (22 мс), т. к. на одну итерацию приходится всего ~0,24 мс работы и затраты на создание параллельной области и сложение буферов на каждой из K = 100 итераций становятся заметными. С ростом d выигрыш SIMD растёт (serial/SIMD: 1,05 при d = 8 и 1,57 при d = 256): чем длиннее внутренний цикл, тем меньше доля накладных расходов на строку (хвост цикла, свёртка частичных сумм в одно число), при d = 8 это всего 2 вектора по 4 double.

2–3. На 2 потоках OpenMP ускоряется в 1,5 раза, OpenMP+SIMD — в 1,3 раза относительно своих однопоточных версий: векторная версия уже быстрее считает, и доля времени на чтение X из памяти у неё больше. Ускорение меньше числа потоков, потому что каждая итерация заново читает всю матрицу X (25,6 МБ при $N = 10^5$, около 0,5 FLOP на байт — задача ограничена памятью), а также из-за последовательных частей: обновление w и создание параллельной области на каждой итерации. Ядер всего 2, поэтому после 2 потоков ускорение почти не растёт (1,55–1,75 в пределах разброса замеров на виртуальной машине), а эффективность падает с 0,75 до 0,10 при 16 потоках — видно насыщение.

4. См. блок о векторизации: автоматически векторизованы поэлементные циклы и (упорядоченно) скалярное произведение; omp simd разрешил переставить сложения, что дало ускорение 1,25 на $N = 10^5$. Флаги дают больше всего: -O3 ускоряет serial в 3,4 раза, -march=native (AVX2) — ещё в 1,5 раза, omp simd — ещё в 1,25 раза.

5. С ростом λ коэффициенты сжимаются к нулю: при $\lambda = 0$ нулевых коэффициентов нет, при $\lambda = 10^{-4}$–10^{-2} обнуляются ровно 8 «истинно нулевых», а при $\lambda = 0{,}1$ — уже 18 из 32, включая небольшие полезные. Функция потерь растёт (0,0004 → 0,72) из-за штрафа и худшей подгонки; время от λ не зависит.

6. Производительность ограничивают доступ к памяти (поток данных X на каждой итерации), накладные расходы потоков (K параллельных областей с барьерами) и лишь 2 ядра виртуальной машины; объединение частичных градиентов дешёвое (p·d сложений), гонок нет благодаря буферам потоков, выровненным по 64 байта.